Pandas Cheatsheet

Benchmarking Tabular Data Augmentation — thesis pipeline reference

Task	Code	Returns	Notes
Load CSV	`pd.read_csv("file.csv")`	`DataFrame`	Add `na_values=["?"]` for dirty CSVs
Load sklearn dataset	`load_iris(as_frame=True).frame`	`DataFrame`	No download needed
Load OpenML dataset	`fetch_openml("name", version=2, as_frame=True).frame`	`DataFrame`	Always pin `version=` for reproducibility
Row + column count	`df.shape`	`(int, int)`	Check this first on any new dataset
Types + null counts	`df.info()`	printed summary	Single best first-look command
Numerical stats	`df.describe()`	`DataFrame`	Reveals outliers before scaling
Type per column	`df.dtypes`	`Series`	Text columns appear as `object`
Missing per column	`df.isna().sum()`	`Series`	Run before any preprocessing
Class distribution	`df["target"].value_counts()`	`Series`	Imbalance affects NMI/ARI interpretation
First / last rows	`df.head()` / `df.tail()`	`DataFrame`	—
Single column	`df["col"]`	`Series`	—
Multiple columns	`df[["a", "b"]]`	`DataFrame`	Double brackets required
Rows by position	`df.iloc[0:5]`	`DataFrame`	End index is exclusive (like numpy)
Rows + cols by position	`df.iloc[0:5, 0:2]`	`DataFrame`	—
Rows by label	`df.loc[0:5, "col"]`	`DataFrame` / `Series`	End index is **inclusive**
Boolean filter	`df[df["col"] > 6]`	`DataFrame`	Most common selection pattern
Combine conditions	`df[(df["a"] > 6) & (df["b"] == 2)]`	`DataFrame`	Use `&` / `|`, never `and` / `or`
Numerical columns	`X.select_dtypes(include="number").columns.tolist()`	`list`	Call on `X`, never on full df with label
Categorical columns	`X.select_dtypes(include=["object","category"]).columns.tolist()`	`list`	Call on `X`, never on full df with label
Drop rows with NaN	`df.dropna()`	`DataFrame`	Risk losing rare classes — prefer fillna in benchmarks
Fill NaN with mean	`df["col"].fillna(df["col"].mean())`	`Series`	Safe default for numerical columns
Fill NaN with median	`df["col"].fillna(df["col"].median())`	`Series`	Better when outliers are present
Fill NaN with mode	`df["col"].fillna(df["col"].mode()[0])`	`Series`	Use for categorical columns
Fill all numerical NaN	`df.fillna(df.mean(numeric_only=True))`	`DataFrame`	Do this before `fit_transform`
Rename columns	`df.rename(columns={"old": "new"})`	`DataFrame`	Does not modify original
Drop a column	`df.drop(columns=["col"])`	`DataFrame`	Add `errors="ignore"` for safety
Add derived column	`df["new"] = df["a"] ** 2`	—	Modifies original in place

Task	Code	Returns	Notes
Convert type	`df["col"] = df["col"].astype(float)`	—	Fix `object` columns rejected by StandardScaler
Split features	`X = df.drop(columns=["target"])`	`DataFrame`	Always by name — position is fragile
Split label	`y = df["target"]`	`Series`	Never passed to the preprocessor
Unique class labels	`np.unique(y)`	`ndarray`	`len(np.unique(y))` gives `n_clusters` for k-Means
Preprocess (fit)	`preprocess.fit_transform(X)`	`ndarray float64`	Training set only — use `.transform()` on new data
DataFrame to numpy	`df.to_numpy()`	`ndarray`	Drops index and column names
numpy to PyTorch	`torch.from_numpy(arr.astype("float32"))`	`Tensor`	Cast to float32 — sklearn outputs float64
Save results	`df.to_csv("file.csv", index=False)`	—	`index=False` keeps the file clean
Append one row	`row.to_csv("f.csv", mode="a", header=False, index=False)`	—	Crash-safe — persists each run immediately
Load results	`pd.read_csv("results.csv")`	`DataFrame`	—
Summarise by group	`df.groupby(["dataset","aug"])[["nmi","ari"]].mean()`	`DataFrame`	Produces the final thesis results table